

Good. I've inspected the **Monevo-main.zip** you uploaded. We can move from the prototype into the real Monevo architecture now.

Your current project is:

- React 19
- TypeScript
- Vite
- Dexie/IndexedDB
- Local-first storage
- Currently only **Profile**, **Account**, and **Transaction** models
- Most of the financial logic is still inside **App.tsx**

So I recommend **not adding more features directly into **App.tsx****. We should first restructure the foundation.

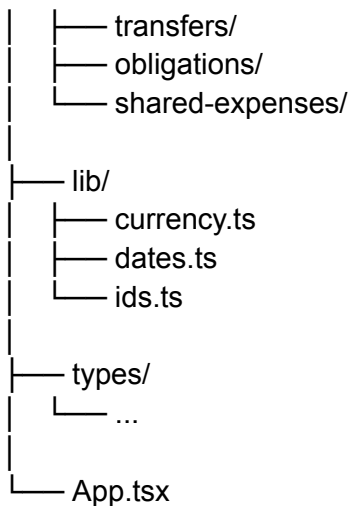
Monevo — Phase 1: Core Financial Engine V2

This is our next build.

1. New architecture

We'll move toward:

```
src/
├── app/
│   └── ...
├── components/
│   ├── accounts/
│   ├── transactions/
│   ├── transfers/
│   ├── obligations/
│   └── shared-expenses/
├── db/
│   ├── database.ts
│   ├── schema.ts
│   └── migrations.ts
├── domain/
│   ├── accounts/
│   └── transactions/
```



We don't need to create all of those folders immediately. We'll introduce them as each feature becomes necessary.

2. First database model

The current:

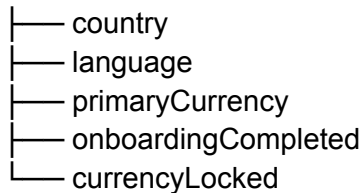
Account

Transaction

becomes a proper financial model.

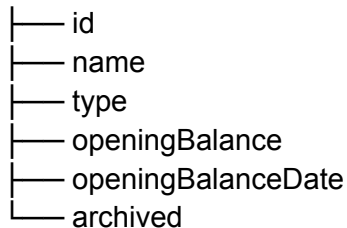
Profile

Profile



Account

Account



But the important change is:

Opening balance is not the current balance.

3. AccountMovement

This becomes the financial engine.

AccountMovement

- |— id
- |— accountId
- |— amount
- |— financialDate
- |— eventType
- |— sourceId
- |— createdAt
- |— reversedBy

Example:

You have:

Bank

Opening balance = AED 2,000

Then:

Expense = AED 100

Ledger:

Opening balance	+2000
Expense movement	-100

Current balance	+1900
-----------------	-------

Another expense:

-50

becomes:

2000

-100

-50

1850

The account balance is **derived**, not manually edited.

That is one of the most important architectural decisions in Monevo.

4. Signed balances

We'll fix something in your current code.

Right now this exists:

```
if (!Number.isFinite(balance) || balance < 0)
```

That means negative opening balances are impossible.

But Monevo's specification says account balances are signed.

So:

Bank

AED +2,500

means money available.

Credit card

AED -1,200

means money owed.

This lets us distinguish:

You owe

from:

Account debt

which is essential later.

5. Transaction becomes an event

Instead of the transaction itself being responsible for the account balance:

Transaction



AccountMovement

For example:

Expense

AED 75

Food

Bank

creates:

Transaction



AccountMovement

Bank -75

The transaction describes **what happened**.

The movement describes **what happened to the account**.

That separation will save us a lot of problems later.

6. Transfers

We'll then add:

Transfer

- id
- fromAccountId
- toAccountId
- amount
- financialDate
- description
- status

Example:

Bank → Cash

AED 500

creates:

Bank -500

Cash +500

But:

Income = 0

Expense = 0

This prevents one of the most common problems in finance apps.

7. Transaction dates

Currently every transaction automatically uses:

financialDate: today()

We'll change that.

The user will choose:

Date

But completed financial transactions cannot be future-dated.

So:

Yesterday ✓

Today ✓

Tomorrow ✗

Technical timestamps remain separate from financial dates.

8. Transaction details

The new transaction form will eventually be:

Type

Expense / Income

Amount

AED 125.50

Date

24 September 2026

Account

Main Bank

Category

Food

Merchant / Person

Carrefour

Description

Weekly groceries

Notes

Optional

For income:

Income

AED 3,050

Account

Main Bank

Description

September salary

No fake “salary category” is required at this stage.

9. Categories

We'll replace the hard-coded array:

```
const categories = [...]
```

with database-backed categories.

Default:

Food

Transport

Housing

Entertainment

Health

Shopping

Other

Then users can add:

Gaming
Family
Church
Subscriptions
Side Hustle

Categories can be archived but not destroyed if historical transactions reference them.

10. Contacts

Then we'll introduce:

Contact

with:

Person
Merchant
Company
Other

So instead of:

description: "Alvin owes me"

we can have:

Contact
Alvin

Obligation
AED 683

Shared Expense
AED 100

That becomes extremely powerful once we reach obligations.

11. What the user will see after Phase 1

The app should feel roughly like:

MONEVO

September 2026

Income	AED 3,050
Spent	AED 1,420
Net	AED 1,630

Accounts

Main Bank	AED 2,180
Cash	AED 350
Credit Card	-AED 620

Recent activity

24 Sep	
Groceries	-AED 85
23 Sep	
Salary	+AED 3,050
22 Sep	
Transport	-AED 25

And underneath the interface, the ledger is already structured properly.

Then we build outward

Once this foundation passes testing:

CORE LEDGER

↓
Transfers
↓
Contacts
↓
Obligations
↓
Repayments
↓
Shared Expenses

